

Company: PhonePe

Job Profile:

Offer Type :Intern+FTE- Software Development Engineer (SDE)

Internship Stipend: 80K

CTC: 33 L CTC (19 Base + Stocks + Others) + 5 lakh joining bonus

Location: Bangalore / Pune

Process Date: 7-8 July 2025

Placement Process:

1. Online Assessment [200+ Candidates , 7th July]

The Online Assessment was conducted on 7th July 2025 via the DOSelect platform. It was a 90-minute test originally scheduled to begin at 4:00 PM but was delayed due to technical issues, and my test eventually started around 4:40 PM. Despite the delay, I stayed calm and maintained a composed attitude. I proactively communicated with the DOSelect support team to resolve the issue. Interestingly, the same individual who assisted me during the technical glitch later appeared in my HRM interview, and I believe my calm response left a positive impression.

Before the OA began, a pre-placement talk was held from 2:30 PM to 3:30 PM. I made it a point to take notes during the session, especially on important points such as the company's vision, founding inspiration, growth metrics, and their underlying technologies and architecture like microservices and scalability. These notes proved useful in later interview rounds, particularly when I had to refer to PhonePe's infrastructure, scalability, and database systems.

All students who applied were allowed to take the OA, and there was no shortlisting prior to the test. Shortlisting after the assessment was based on problem-solving accuracy, the efficiency of code, the number of passed test cases, and behavioral logs captured by DOSelect's secure browser, which monitors for tab switches, fullscreen exits, and suspicious activity.

Test Structure & Questions

● 1. Friendship Clusters

Problem: Given a list of integer pairs representing friendships, determine the minimum number of additional friendships required to make all people directly or indirectly friends with each other.

If person A is friends with person B, then all their friends are also friends.

Approach:

- Modeled it as a graph.
- Used DFS to find the number of connected components (disjoint sets).

- Final answer = number of disjoint sets - 1.
 - Issue: Got a stack overflow on one large test case due to recursion depth but chose to move on since I had to manage time.
 - Could have used DSU
-

- **2. Array Assignment with Max Difference Constraint**

Problem: Given an array with some pre-assigned elements and a constraint k , find the number of valid ways to assign the remaining elements such that the difference between adjacent elements is at most k . Each element must be between 1 and 100.

Approach:

- Used recursive backtracking with memoization.
 - Solved 7 out of 12 test cases.
-

- **3. Pair with Bit Position Sum Constraint**

Problem: From an array, pick two numbers (a pair) whose sum is exactly s such that the total of positions of set bits in both numbers is less than or equal to k . Count the number of such pairs.

For example, consider the number 12. Its binary representation is 1100, and the set bits are at positions 2 and 3 (starting from position 0 on the right). So, the **sum of bit positions** is $2 + 3 = 5$.

Approach:

- Brute-forced through all pairs.
 - Calculated set bit positions using bit manipulation.
 - All test cases passed except one.
-

- **4. House Assignment with Friend Constraints**

Problem: Given friend pairs and their house preferences (4 houses with limits), assign each student a house such that:

- Friends don't get the same house
- House limits are respected

Input Challenge: Input was provided as a complex nested list (e.g., $[[h1, h2], [h2, h3]]$) with no clear count of friend pairs.

- **Approach:**

- Due to lack of time and complexity of parsing, printed "possible" as a fallback solution.
- Managed to pass approximately 60% of the test cases.

During the test, I faced a Wi-Fi issue and had to switch networks. Since DOSelect tracks activities like fullscreen exits and network changes, I immediately called the representative to inform them and ensure they witnessed the switch. I also followed up with the POC to document the situation properly. Transparency during such incidents is crucial, as it helps protect against being falsely flagged for violations.

The test results were released around 9:30 PM, leaving me with very little time to revise or prepare. We were asked to report the next morning by 8:00 AM, so I had to make the most of the limited time. While traveling in the local train and metro, I made sure to quickly go through some previously asked problems and revise key topics like graphs.

Additionally, it's important to groom yourself well before the interview. Dress professionally—wear formal attire—as it helps create a good first impression and shows that you're taking the process seriously.

Round 2 – Technical Interview (First Tech Round) [30 students]

I reached campus at 7:30 AM. At 8:30 AM, interview batches were announced; I was in the second batch. The first batch started at 9:00 AM, so I had an hour to calm down and prepare. At 10:00 AM, my turn came. I knocked, entered, shook hands with the interviewer, who introduced himself as an App Developer at PhonePe. I gave a brief self-introduction, then the interview started.

Question 1: [Minimum Number of Operations to Sort a Binary Tree by Level](#)

He opened with a medium-level tree problem and asked me to explain the logic before jumping into code.

I explained that I would do a **BFS traversal** of the tree, level by level. For each level, I'd collect all the node values into a list. Then, I'd make a sorted copy of that list. Using a map of value $\rightarrow$ index from the original list, I would compare the original and sorted arrays index by index. If the values at index i didn't match, I'd swap them in the original array and increment a counter. I would also update the map after every swap. This technique is similar to solving the **minimum swaps to sort an array** problem.

He listened carefully and seemed satisfied. After a few clarifying questions, he asked me to write the code and then discuss the **time complexity**. I did both. Although he mentioned he was expecting a graph-based approach, he acknowledged that my solution was valid and handled the problem efficiently. We had a brief back-and-forth on potential optimizations and edge cases.

Question 2: [Find the K-th Smallest Sum of a Matrix With Sorted Rows](#)

This was a tough one. I started with a **brute-force** approach: generate all possible sums by combining one element from each row and storing the results in a **max-heap** of size k . He immediately asked why I was using a max-heap instead of a min-heap.

I explained my reasoning — that the max-heap would help keep track of the k smallest sums by discarding the largest as new smaller ones come in. He looked a bit surprised but agreed with my logic once I did a **dry run** of the approach.

Then he asked me to **optimize** the solution. I took a pause to think but couldn't come up with an efficient algorithm on the spot. *I admitted that I was stuck and asked for a hint.* He kindly walked me through a more optimized method.

He then asked whether **duplicates** could occur in the sums. I answered yes. He asked why, and I gave a correct explanation. When he asked how to **avoid duplicates**, I suggested using a **HashSet** to track visited combinations. He nodded and seemed happy with that answer.

Post-Interview Conversation

After the technical questions, he asked if I had any questions for him. I asked about a couple of **app features** and their **color palette choices**—he smiled and admitted those weren't part of his responsibilities but still gave me general answers. Then I asked him about their **tech stack**, and he shared that they had recently started using **React Native**. Before I could ask why, he explained the reasoning behind the shift in detail, which I found really insightful.

Post-Interview Reflection

The interview ended around **11:00 AM**. Although I was relieved it was over, I didn't feel too confident, especially about the second question. I overheard a few other candidates saying they had solved it, which made me a bit anxious.

The results were expected soon, but they didn't come until **3:00 PM**. During this waiting period, some candidates were called for **Technical Round 2**, while others moved on to **HR interviews**.

To keep my nerves in check, I spent the time revising **System Design** concepts, casually discussing things with friends, and grabbing some refreshments. Those small breaks helped me ease the pressure and clear my head for whatever was coming next.

3. Round 3 – Technical Interview (Final Tech Round) [Around 11 students]

Time: Around 3:00 PM onward

Duration: Approx. 1 hour

By **3:00 PM**, I had gotten quite relaxed after the long wait. I made sure to stay composed and walked into the interview room calmly. I knocked, asked for permission, and entered. The interviewer greeted me with a handshake and introduced himself. He mentioned that he works on **KYC**, **user verification**, and **database systems** at PhonePe.

I introduced myself in a similar way, giving a brief about my background, skills, and a short description of one of my projects. Without wasting any time, he jumped straight into the first question.

Question 1: Lava Brick Heat Loss in Matrix

Problem Description:

Given a matrix where each element represents the heat removed from a lava brick when it passes through that cell. Starting at position $(0, 0)$, you must reach $(n-1, m-1)$ with **minimum total heat loss**.

- At any point, you must move **exactly 3 steps in one direction (up, down, left, right)**.
- After completing those 3 steps, you **must immediately change direction** (no continuing in the same direction).
- You **cannot reverse** direction.

My Approach:

I immediately recognized that the problem could be treated as a **graph traversal** problem.

- I considered each **cell as a node**, and the **edges** as moves with the weight equal to heat loss.
- I suggested **Dijkstra's Algorithm** and explained in detail about the characteristics of **Dijkstra** and why it was suitable for this weighted graph problem.
- I also highlighted the **need for a modified state**: `i, j, direction, steps_remaining, total_weight`.

He asked me to explain it visually, so I first drew a **standard Dijkstra graph**, showed how it processes nodes, and then extended it to demonstrate why additional state parameters (like direction and steps) were necessary in this case. I justified this as expanding the graph to a more detailed representation of possible paths.

Code Implementation:

He then asked me to write the code. While implementing:

- I paused to ask him a few clarifying questions regarding direction changes and boundaries, and he was supportive in answering.
- I realized I had missed the **relaxation condition**, so I asked if I could write a generalized version elsewhere in the code. He agreed.
- Some directional code blocks were repetitive, so I asked if I could omit them with comments for clarity. He agreed again.

I then completed the code with detailed comments and added placeholders for repeated blocks and the generalized relaxation. After that, he asked me to explain the code again and discuss the **time and space complexity**, which I did thoroughly. He verified the logic and moved to the second question.

Problem Setup:

He presented the problem in an **incomplete form (as I understood it)**, giving only a partial description and asking me to proceed with a solution based on it. I kept asking for clarification, but he insisted that this was all the information I would get.

My First Attempt:

I assumed a possible setup and gave him a brute-force-style solution. For the sample test case, it passed. However, he then gave me another test case where my approach failed. I revised my logic and proposed another solution. *He pointed out that it would also fail and asked me to come up with a counter-example that would break my logic.*

I took a moment to think and asked if I could get a glass of water. He nodded. I joked, “Will this disproving of my own solution count?” He smiled and said, “Yes, yes,” while laughing. Eventually, I did manage to find a test case where my solution would break.

At this point, he provided the **full problem statement**, mentioning that it involved **dynamic programming**. I initially struggled to grasp the direction of the solution, so I asked for the **constraints**. Once I had them, I was able to identify the DP approach.

My Final Solution:

I explained a **recursive + memoized** solution, wrote the code, and discussed both **time and space complexity**. I also explained how the solution could be converted into a **bottom-up (tabular)** DP formulation.

Post-Interview Interaction

Once the technical questions were done, he asked if I had any questions for him. I asked about:

- The **challenges faced in KYC and verification systems**
- What is that random message which gets sent when someone links their UPI/Bank?
- His **most impactful contribution** to the PhonePe ecosystem

He answered each of them in detail, sharing both technical and team-level insights.

Before I left, he smiled, shook my hand again, and said:

“You’re the most calm and relaxed person I’ve spoken to today.”

With that, the interview ended on a positive and memorable note.

Round 4 – Hiring Manager Interview

Time: Just 10 minutes after Round 3

Interviewer: Same person who handled my technical issue during the Online Assessment (OA)

I wasn't sure if he remembered me from our earlier interaction during the OA when I had technical issues, but he greeted me with a smile and a handshake. Without much small talk, he asked me to introduce myself.

Self-Introduction and Resume

I gave the same structured introduction I'd prepared—covering my name, background, skills, and a quick summary of my project. He asked me if I was tired to which I replied yes but equally excited.

He scanned my resume for about a minute and then asked me to **pick the most exciting and challenging project** and walk him through it.

I began by explaining:

- What the project was about
- The **goal and motivation** behind it
- The **tech stack** used
- My specific **role** in the team

When I mentioned that my role involved designing **model schemas**, he immediately asked me to **list down the tables and their schemas**. I admitted I couldn't recall all of them in full detail but shared the key tables and schema structure from memory.

He then asked me to walk through the **user flow** in the project. Since the project involved **GenAI**, he had several follow-up questions on how I integrated it. I explained the **RAG (Retrieval Augmented Generation)** architecture and how I approached **prompt tuning**. I also admitted that my tuning approach wasn't based on any specific methodology—it was entirely experimental and observation-driven.

He asked what **hallucinations** in GenAI meant, and I explained that too.

API Design & Project Ownership

He then asked about a few **specific APIs** used in the project. I clarified that I didn't personally implement those but had a rough understanding. I even drew a **high-level diagram** of how I thought the flow might look.

To show stronger ownership, I brought up another project I worked on using **MERN stack**, where I had built the APIs myself. I explained:

- API structure
- How user authentication worked
- Use of **JWT tokens** and **hash functions**

He then asked, "Where is the hash code stored?" I answered "in the virtual environment," but he pointed out that this would be a **security issue**, which I acknowledged, adding that I hadn't deeply explored that side yet.

System Design: YouTube

Next, he asked about my **most used app**. I said **YouTube**, and he asked me to explain **how its APIs might work**. I confirmed whether he wanted a **project-level view** or a **system design-level view**, and he chose the latter.

I started explaining:

- **DNS lookup**
- **API gateway**
- **Load balancing**
- **Service-based architecture**
- **Horizontal scaling**

He then asked how I would handle **video feeds**. I explained that:

- Videos would be delivered in **chunks**
- Additional content would load **only when the user finishes scrolling the current chunk**

Then came the key question:

“How would you decide which videos to send?”

I said that would be handled by a **recommendation system**. He challenged me further:

“Assume you’re just launching YouTube—how would you build a basic recommender without ML?”

I proposed a **rule-based scoring system**, explaining that:

- We'd gather **user data** like subscriptions, likes/dislikes, history, location, etc.
- Each video would have associated **tags**
- Based on **parameter weights**, we'd assign a score to each video
- The highest scoring videos would be sent as feed chunks

He seemed satisfied with the logic and the non-ML approach.

Behavioral and Personal Questions

He shifted toward more personal questions:

Q: “How has choosing CSE in SPIT changed your personality?”

I responded:

- I've moved beyond solving only textbook problems to asking **what, why, and how** behind everything
- I now approach problems by fixing first, identifying root causes, then redesigning
- My curiosity has grown, and I challenge myself often
- I introspect and verify facts before raising questions

Q: “What would you do in case of a conflict with a manager or teammate?”

I gave a real example where we had to choose between **Django** and **MERN** for a project. Although we had used Django in a previous winning hackathon, I convinced the team logically to go with MERN this time based on:

- Current requirements
- Ease of debugging
- Flexibility of tech

I emphasized that I prefer **direct communication**, avoid miscommunication, and back my arguments with **facts and logic**.

Q: “How have the past 6 months been for you?”

I highlighted:

- The value of courses like **Machine Learning, Distributed Computing, Blockchain, Business Analytics**
 - How they improved my clarity of concepts and curiosity
 - Every course had a **mini project**, which taught me about deadlines, collaboration, debugging, quick learning, handling people and team-building
-

Questions I Asked the Interviewer

He then gave me the chance to ask questions. I asked:

1. "I referred to points from the pre-placement talk. PhonePe was launched in 2016, what was its founding inspiration?"
2. "Why hasn't PhonePe moved to AWS, Azure, or GCP? How do you handle so many transactions on your own infra?"
3. "How has PhonePe managed to capture such a huge customer base in rural India, despite strong competitors like GPay?"

He gave thoughtful answers and mentioned:

- **Hiring the right people**
- Emphasis on **financial discipline**
- Focus on **long-term survivability and execution**

As the conversation wrapped up, I smiled and said:

“So that saying about pressure turning coal into diamonds is true?”

He smiled back, and then I asked him if he had any feedback for me.

The results came at 7:45 PM. **7 people got selected.**

Useful Tips:

Why I Got Selected & What Helped Me

One of the key things that helped me was consistent **LeetCode practice** over a long period. I didn't just aim to solve problems—I made sure I **understood why certain solutions worked** and why others failed. For particularly tough problems, I sometimes spent **2–3 hours** just breaking them down, revisiting them until I gained full clarity.

After exams, I specifically **revised graph algorithms**, solving the entire graph section on **CSES Problem Set** and also picked some **hard problems on trees and graphs** to stretch my thinking. That gave me a strong base for several questions that came up during the tech interviews.

Another factor that I believe played a big role was my **calm and composed attitude** during the **Tech Round 2** and the **Hiring Manager Round**. I didn't panic, even when I didn't know a solution completely—I asked clarifying questions, discussed trade-offs, and tried to reason through the problem.

Areas I Could Have Improved

Despite the positives, there were definitely areas where I could've done better:

- I **didn't prepare enough for system design**. While I had some basic understanding, I could have been more confident if I had practiced it more through mock interviews or small-scale design exercises.
- I should have **invested more time in revisiting my own projects**. Especially the ones I had mentioned on my resume—some schema-level or implementation-specific details were fuzzy, which could have been avoided with better preparation.

General Tips for Candidates

- THINK OUT LOUD during the interview.
- Solve a lot of DSA . For DSA, start with [neetcode.io](https://leetcode.com) , then move on to Striver's A2Z sheet. I first completed about 15 problems from each topic on [neetcode.io](https://leetcode.com) [Around 2.5 months] . After that, I focused on DP, Arrays, and Binary Search from Striver's sheet [Around 3 months]. For graphs, I followed the YouTube channel <https://www.youtube.com/@codestorywithMIK> , which has many solved questions across topics—definitely check it out. I also solved the CSES graph section and used ChatGPT to get questions combining multiple topics like DP on graphs and graph + bitmasking.
- And as I said, don't just run behind getting the correct solution—explore the wrong solutions also. Wrong answers teach 10x more than the correct ones. Spend good time around each question. Don't be in a race to finish off questions; it's about understanding deeply, not just ticking boxes.
- A correct code/approach in the interview does not guarantee your selection, nor does a wrong one deny you. It's all about how well you are able to explain and articulate your thoughts.

- Revise your resume and projects thoroughly: Be ready to discuss your contributions, system design choices, and technical decisions. Know the why behind each decision, not just the what.
- Stay calm and composed during interviews: Your attitude matters as much as your answers. If you get stuck, ask clarifying questions, explain your thought process, and don't panic.
- Avoid comparing yourself to others: On the interview day, many people will be talking about how their rounds went, what questions they got, or how confident they feel. Don't let it overwhelm you. Stay focused on your own preparation and performance.
- Be humble and helpful: Everyone around you is equally nervous and aiming for the same goal. Be kind, share a smile and help others if they need it. You're not competing against each other—you're just trying to show your best.
- Take notes during the Pre-Placement Talk: Interviewers often refer to company-specific terms, metrics, or philosophies. Jotting them down can give you an edge during interviews.
- Be mindful of your grooming and energy: Wear something you're confident in (formals preferred), get a good haircut, get enough sleep, and arrive on time. First impressions do matter.
- Don't take failure personally: Not every round or result defines your intelligence or hard work. Learn from it and move forward. You only need one opportunity to click.
- Believe. If you're someone who places faith in God, then trust that everything happens in its own time—God has a plan for you.

You can contact me on WhatsApp. +91 9152752492.

Linkedin [Aditya Tiwari - Mumbai, Maharashtra, India | Professional Profile | LinkedIn](#)